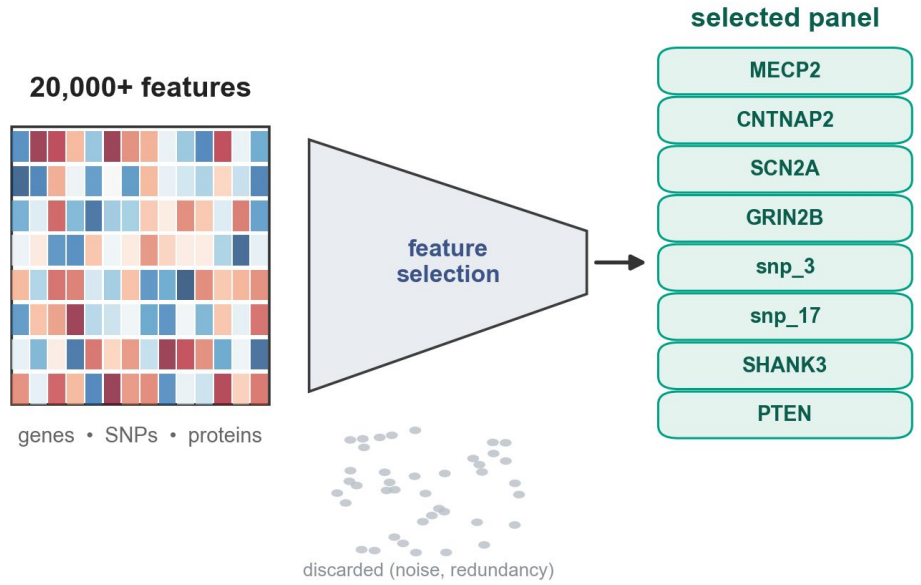
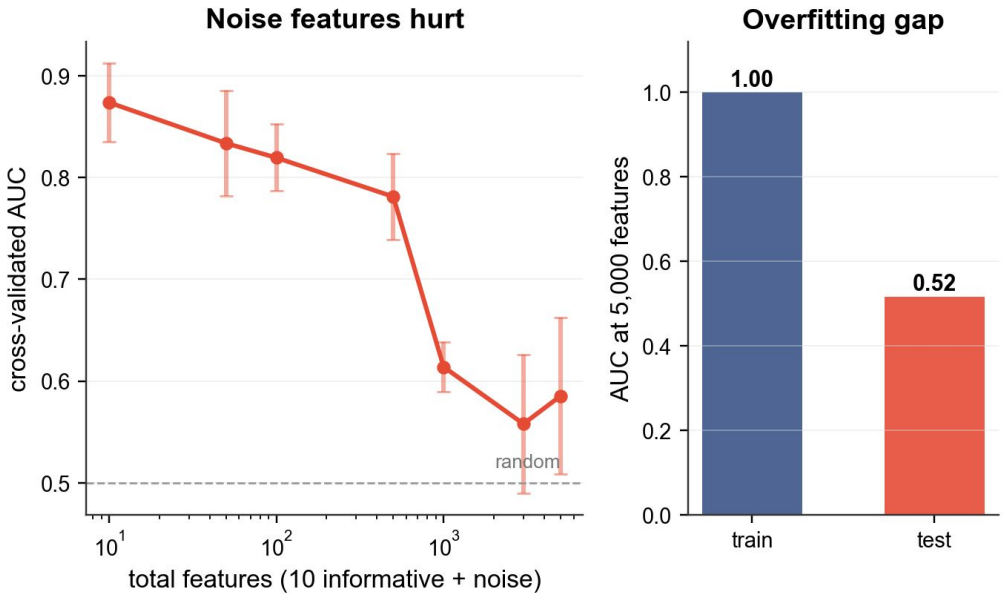
Feature Selection for Multi-Omics

- Lesson 6: Feature selection
- The problem: 20,000 genes + 500K SNPs, a few hundred samples
- Goal: keep the features that carry signal, drop the noise
- Fourteen battle-tested methods, with examples on the ASD data
- Bonus: the leakage trap that invalidates published papers



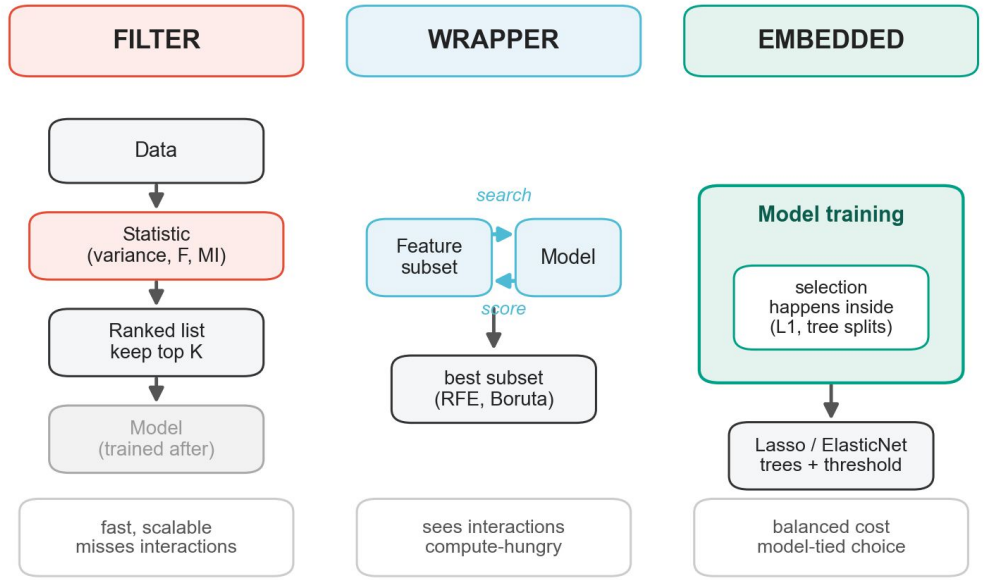
Why Select Features At All?

- Statistical: p >> n → chance correlations, unstable models
- Curse of dimensionality: distances become meaningless
- Practical: faster training, cheaper assays, simpler validation
- Biological: a 15-gene panel is interpretable; 20,000 weights are not
- Clinical: fewer analytes = feasible diagnostic test
- Like packing for fieldwork: take only what you will actually use



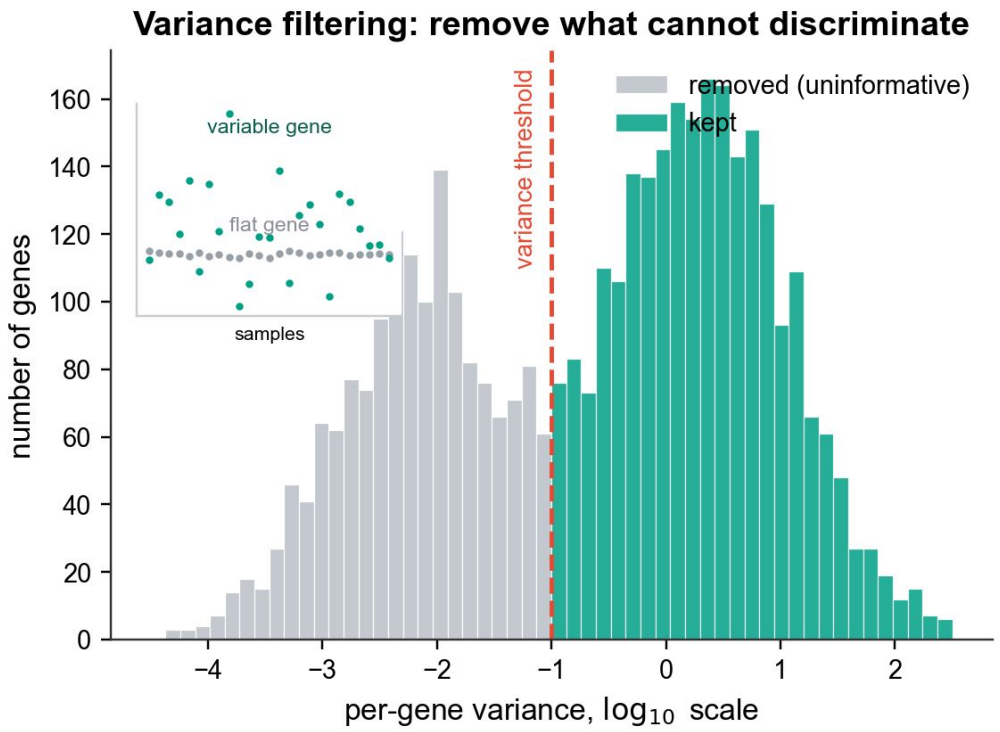
Three Families of Methods

- FILTER: rank features by a statistic, independent of any model
  - (variance, F-test, χ^2, mutual information, correlation)
- WRAPPER: search feature subsets using model performance
  - (RFE, forward/backward selection, Boruta)
- EMBEDDED: selection happens inside model training
  - (Lasso/ElasticNet, tree importances + threshold)



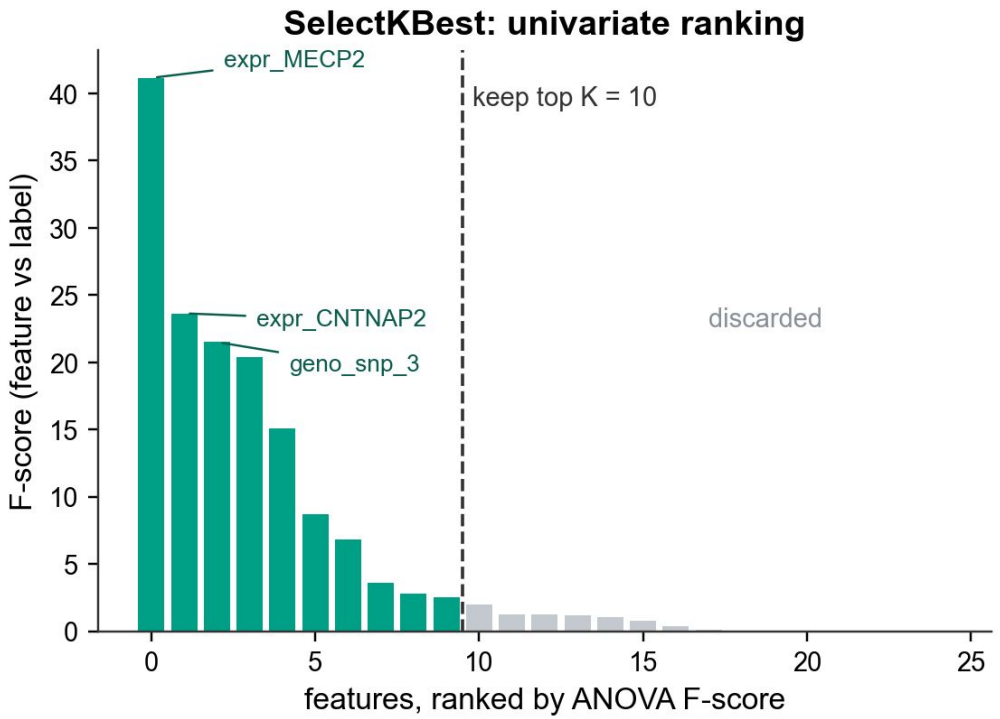
Method 1 - Variance Threshold (Filter)

- Idea: a feature that barely varies cannot discriminate anything
- Remove features with variance below a cutoff
- Catches: unexpressed genes, monomorphic SNPs, constant covariates
- Unsupervised - never sees the labels → zero leakage risk
- sklearn: VarianceThreshold(threshold=0.1)
- Always the first, dumbest, most reliable step



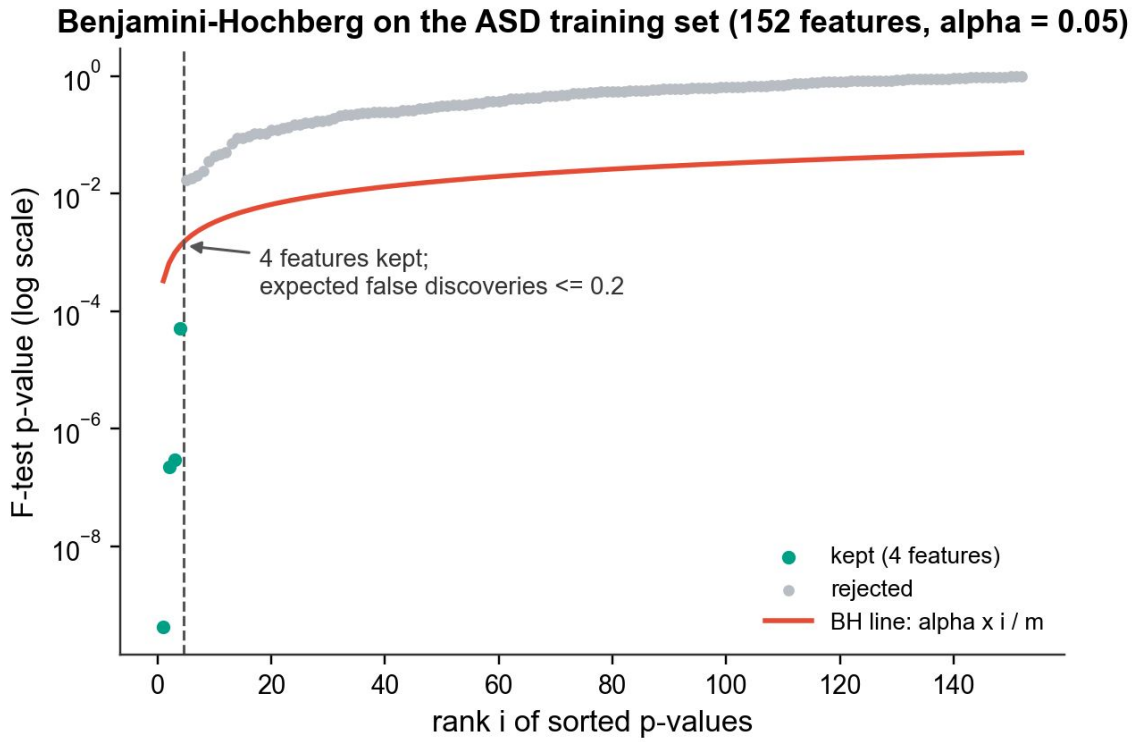
Method 2 - Univariate Tests / SelectKBest (Filter)

- Test each feature against the label, one at a time
- ANOVA F-test: continuous features (expression) vs classes
- sklearn: SelectKBest(f_classif, k=1000)
- χ^2 test: non-negative counts / categorical features
- Keep top K by score, or top percentile SelectKBest(chi2, k=...)
- Fast: 20,000 → 1,000 in seconds; ignores redundancy & interactions



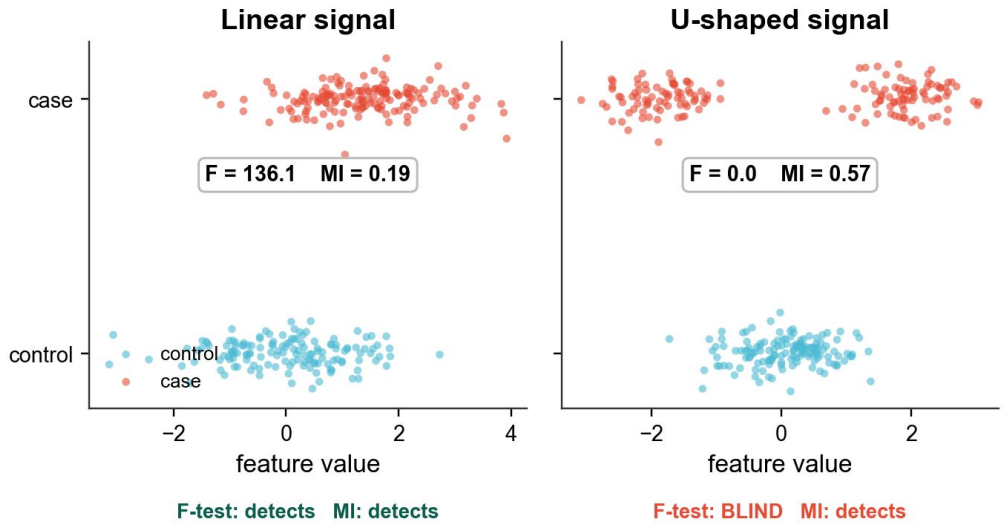
Method 3 - FDR Control: Selecting with Honest p-values (Filter)

- Top-K asks 'how many?'; FDR asks 'how many are REAL?'
- 20,000 tests at p<0.05 → ~1,000 false positives by chance alone
- Benjamini-Hochberg: sort p-values, keep those under the rising line
- FDR = expected fraction of false discoveries among the kept
- sklearn: **SelectFdr**(f_classif, alpha=0.05)
- The same machinery as differential expression - you already use it



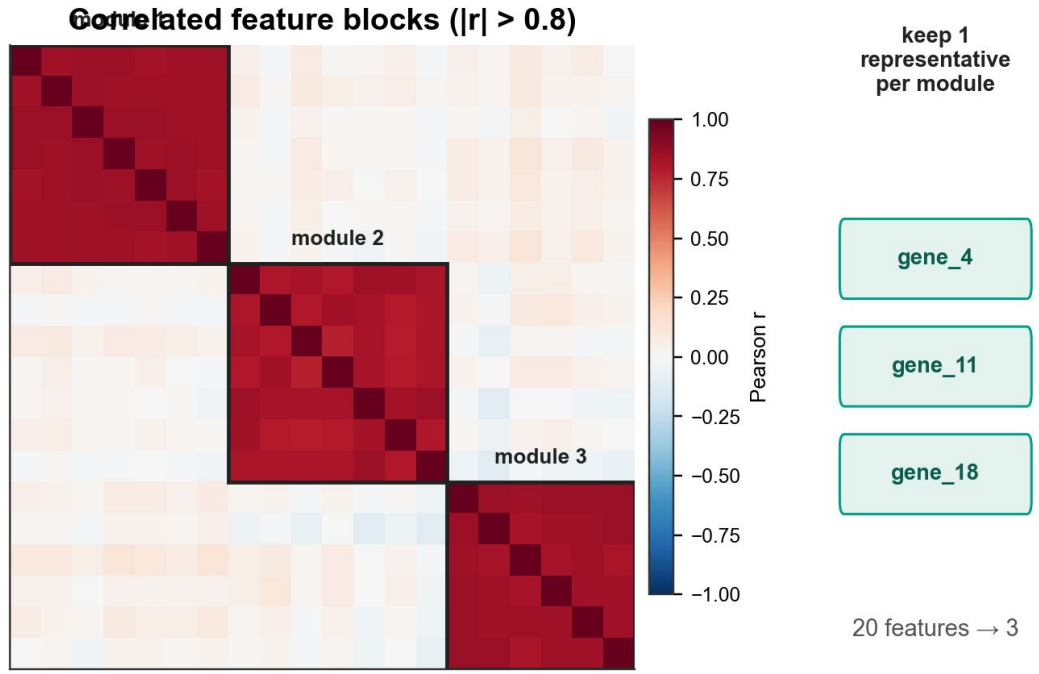
Method 4 - Mutual Information (Filter)

- MI = how much knowing the feature reduces uncertainty about y
- MI = 0 → independent; larger MI → stronger dependence
- Captures NON-LINEAR relations that correlation misses
- Example: U-shaped dose-response scores r ≈ 0, MI high (e.g. gene where BOTH very low and very high expression are pathogenic)
- Needs more samples to estimate reliably than an F-test
- sklearn: SelectKBest(mutual_info_classif, k=1000)



Method 5 - Correlation & Redundancy Filtering

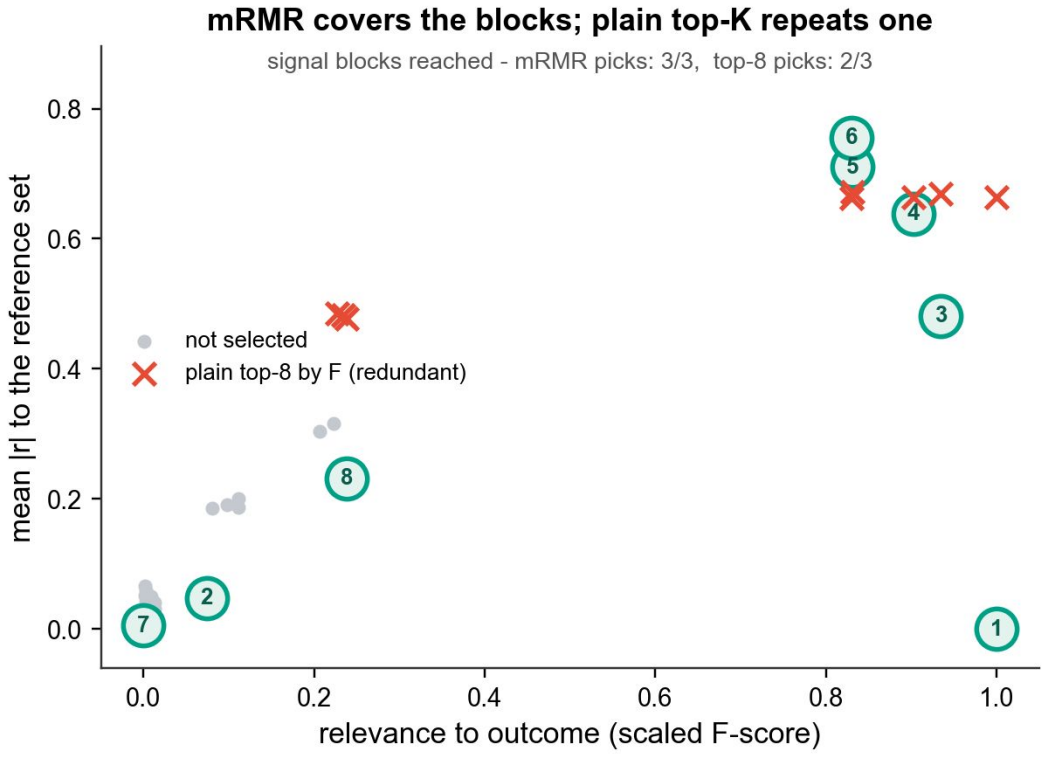
- Omics features travel in packs: co-expression modules
- 50 genes from one module ≈ 1 signal counted 50 times
- Simple: drop one of every pair with |r| > 0.9
- Smarter: mRMR - max relevance to y, min redundancy among features
- Or: select at module level (WGCNA eigengenes, Lesson 3!)
- Redundancy dilutes importance scores across correlated twins



Method 6 - mRMR: Maximum Relevance, Minimum Redundancy (Filter)

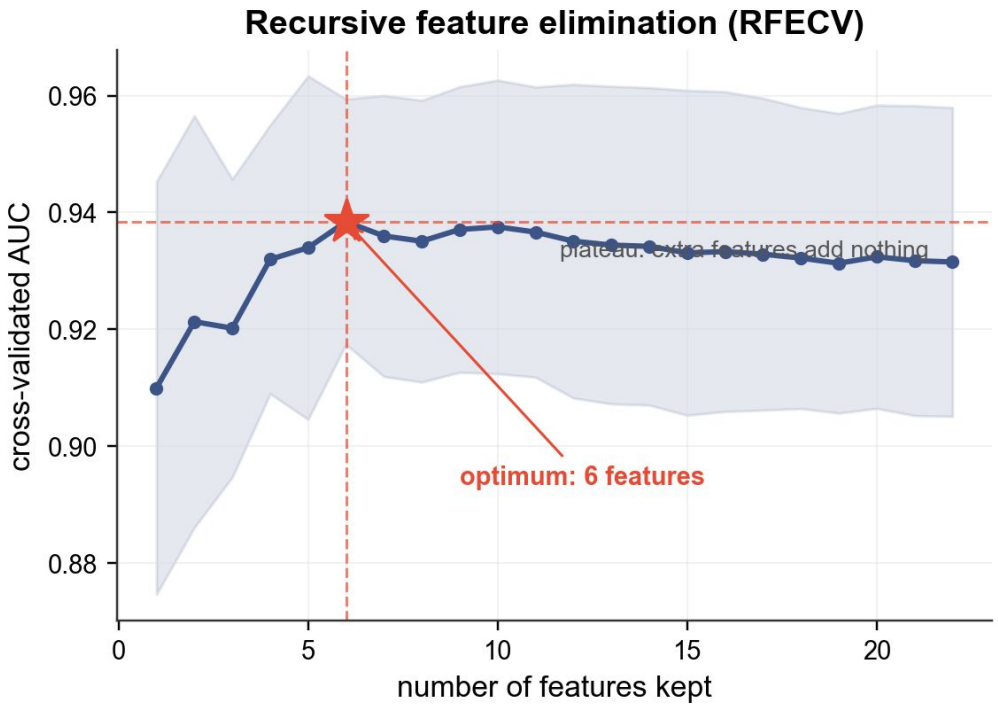
- Top-K keeps 50 copies of the same signal; mRMR refuses to
- Greedy: add the feature with the best (relevance - redundancy) score
  - relevance: association with y; redundancy: mean |r| to already picked
- Classic in omics since microarray days (Peng et al., 2005)
- Small panels that COVER the signal space instead of repeating it

1. Start with the single most outcome-associated feature
2. At every step, score each remaining candidate by its relevance to the label MINUS its average correlation with the features already chosen, and add the best.
3. A gene that is strongly associated with disease but nearly identical to an already-picked module member scores poorly - the panel moves on to a DIFFERENT part of the signal space.



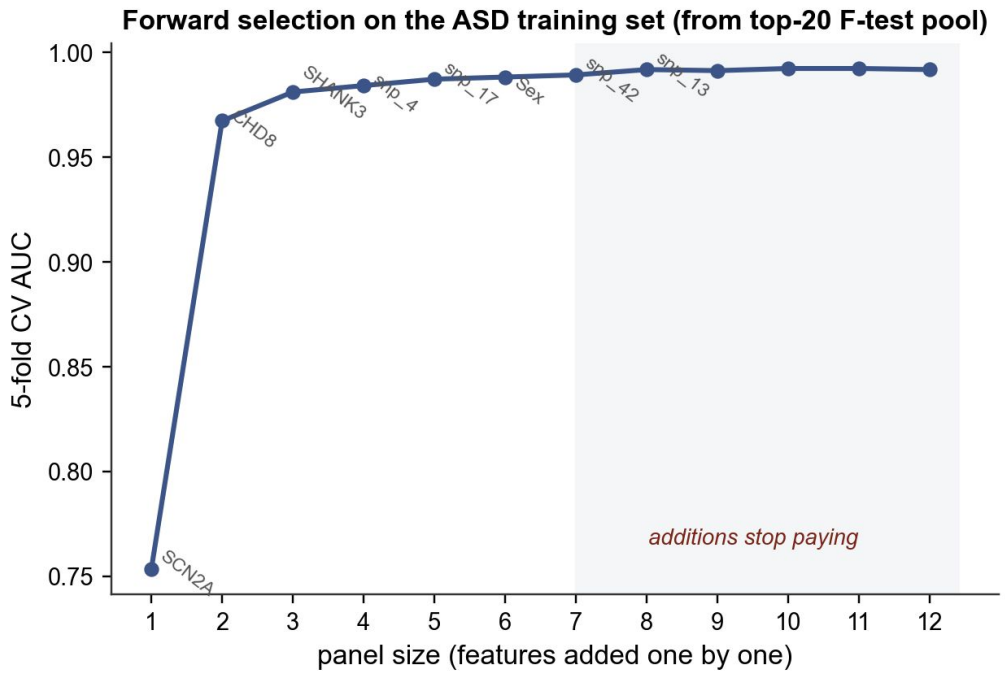
Method 7 - Recursive Feature Elimination (Wrapper)

- Backward elimination driven by a model
- 1) Train on all features 2) rank by importance
- 3) drop the weakest ones 4) repeat until K remain
- RFECV: cross-validation picks K automatically
- Sees features in context - unlike univariate filters
- Reality-TV for features: each round, the weakest goes home



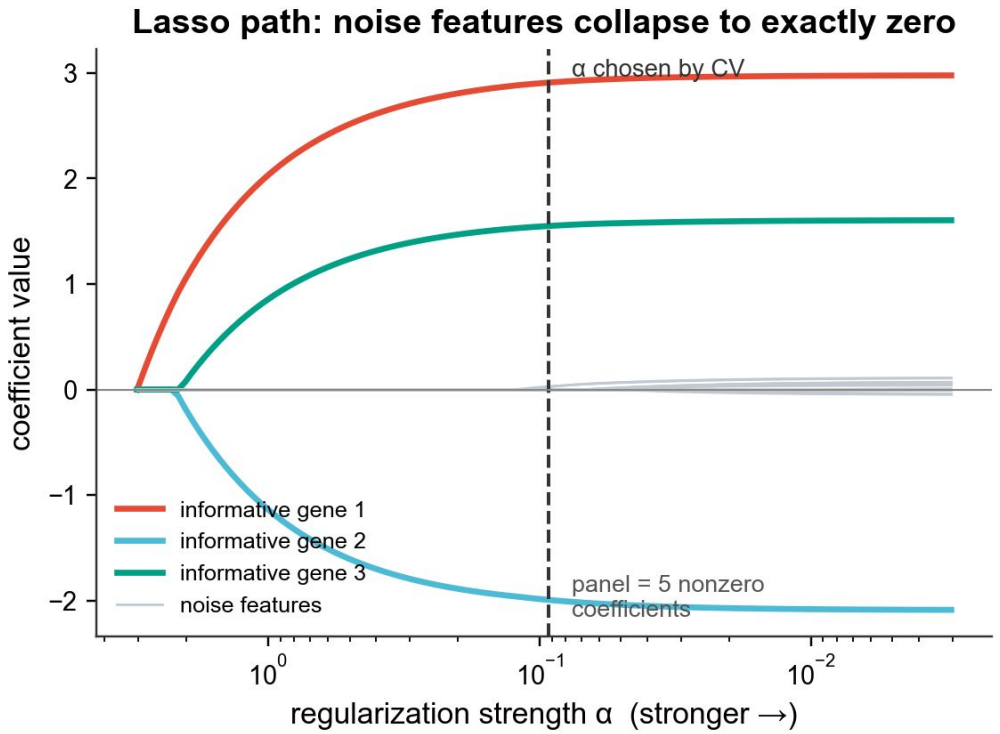
Method 8 - Sequential Forward / Backward Selection (Wrapper)

- Forward: start empty; repeatedly ADD the feature that helps CV most
- Backward: start full; repeatedly DROP the least harmful feature
- Judges candidates by actual model performance, not a proxy statistic
- sklearn: SequentialFeatureSelector(model, direction='forward', cv=...)
- Costs p model fits per step - use after a filter has cut to ~30-100
- Greedy: no regrets allowed - can miss jointly-optimal pairs



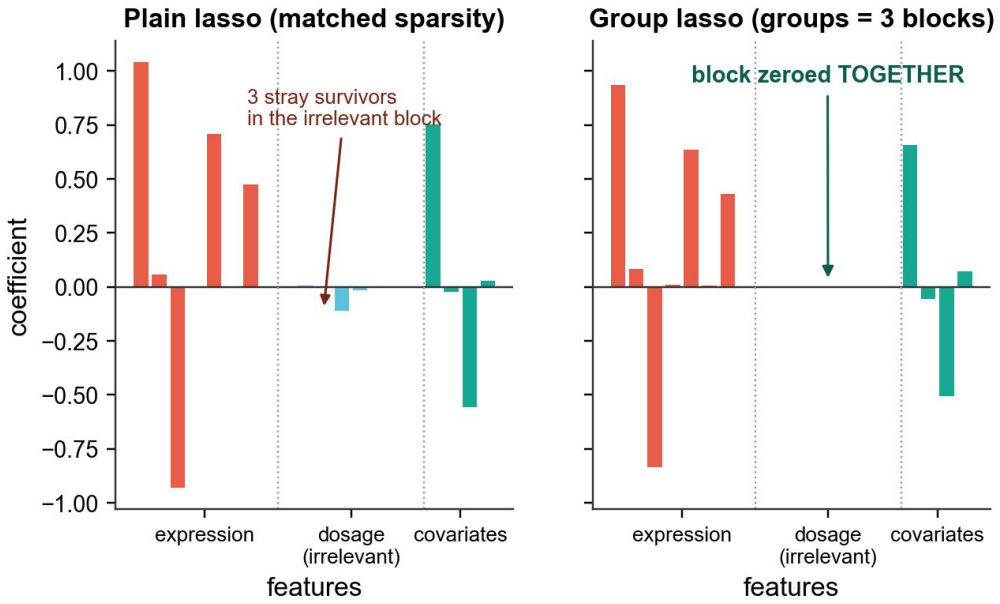
Method 9 - Lasso / ElasticNet (Embedded)

- L1 penalty makes irrelevant coefficients EXACTLY zero
- Training and selection in one step - the model is the filter
- Path: increase α → fewer surviving features
- Lasso quirk: from a correlated group, keeps one arbitrarily
- ElasticNet (L1+L2): spreads weight across correlated genes → stabler
- sklearn: LogisticRegression(penalty='l1'), ElasticNetCV



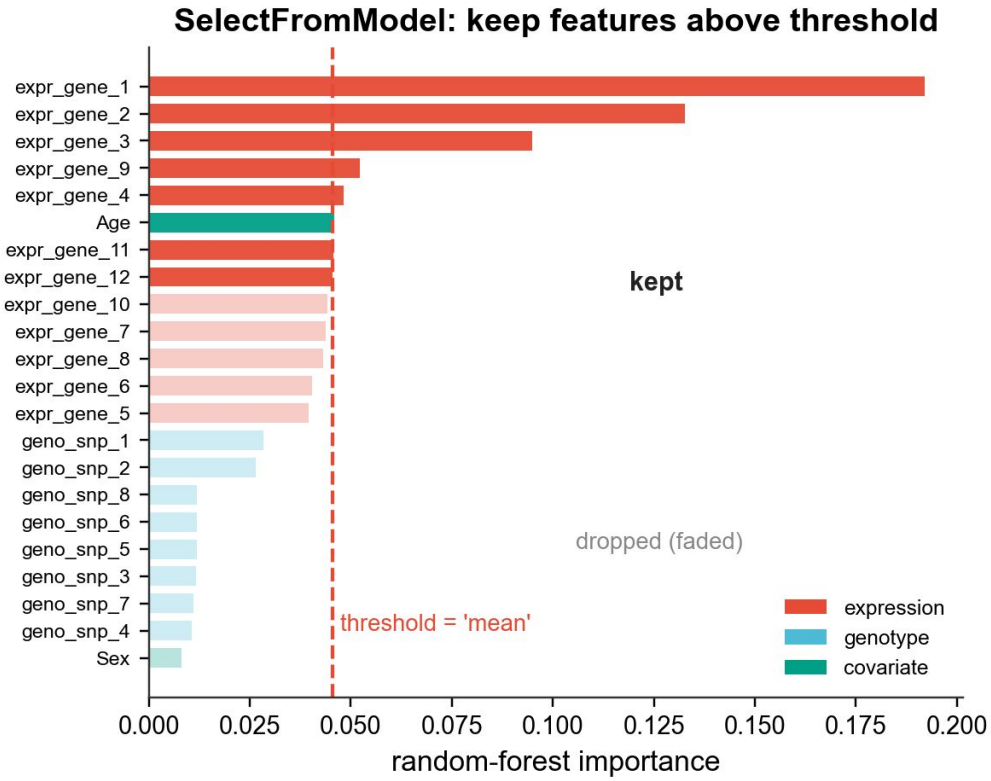
Method 10 - Group Lasso: Selecting Whole Blocks (Embedded)

- Omics features come in natural GROUPS: blocks, pathways, modules
- Penalty on each group's joint norm → groups zero out TOGETHER
- Answers block-level questions: does genotype add anything to expression?
- Sparse-group variant: sparsity between AND within groups
- Notebook: 20-line proximal-gradient implementation on the 3 ASD blocks



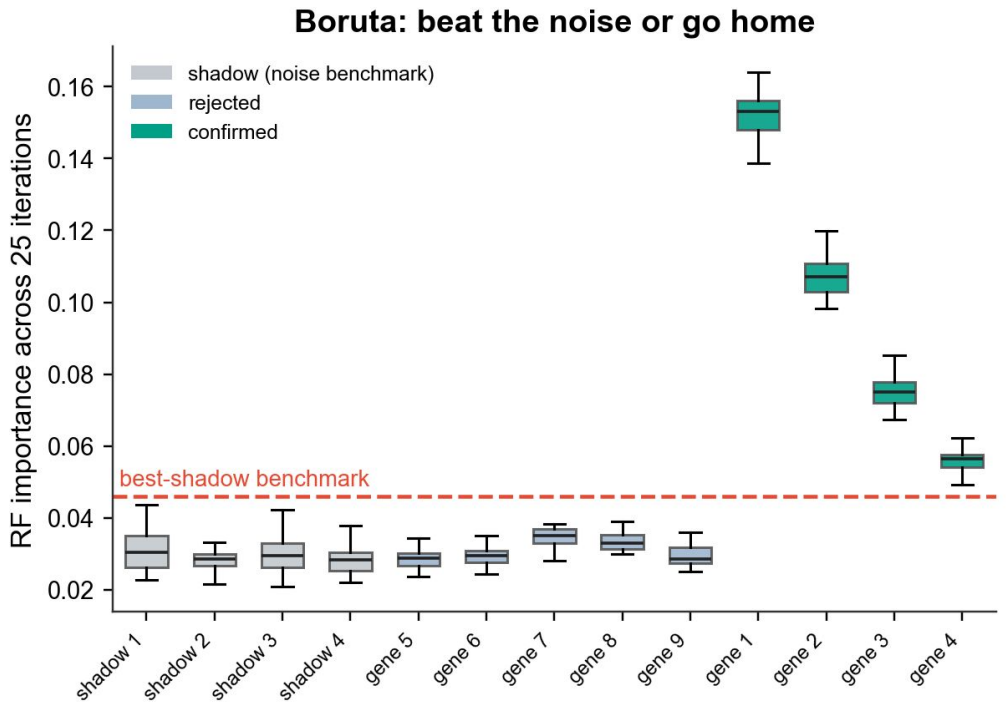
Method 11 - Tree-Based Selection (Embedded)

- Random Forest / XGBoost rank features while training
- SelectFromModel: keep features above an importance threshold
- Thresholds: 'mean', 'median', or top-K
- Captures non-linearities and interactions natively
- Caveat: importances diluted across correlated features
- Caveat 2: biased toward high-cardinality features



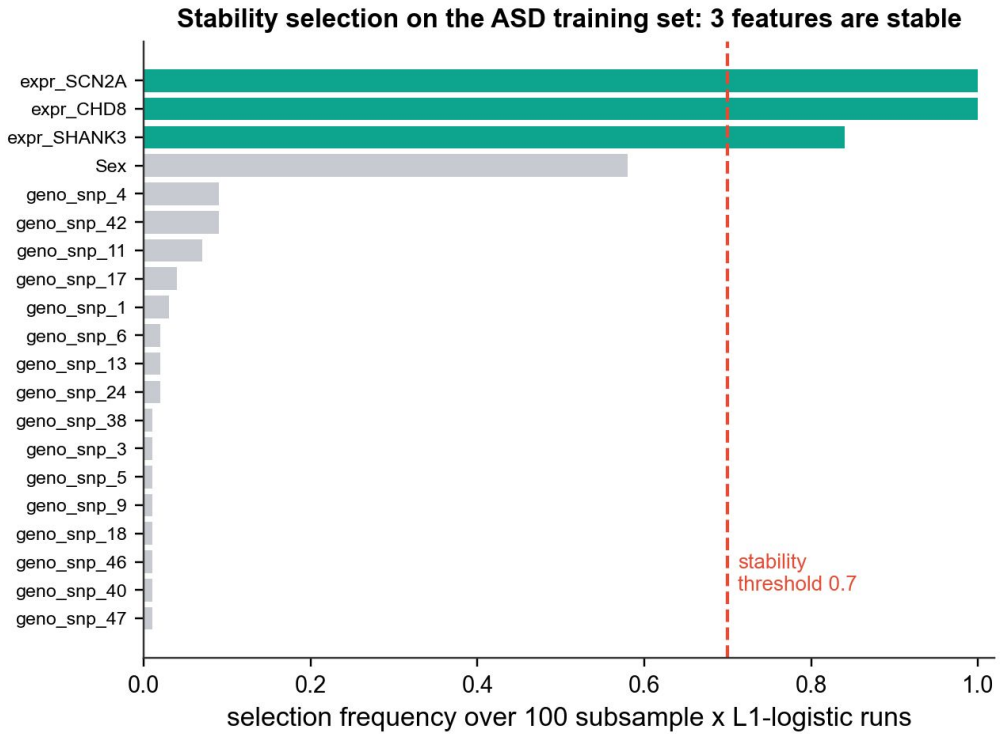
Method 12 - Boruta (Wrapper)

- Question: is this gene better than RANDOM NOISE would be?
- Create 'shadow features': shuffled copies of every real feature
- Train RF on real + shadows; benchmark = best shadow importance
- Features beating shadows repeatedly → confirmed (binomial test)
- ALL-relevant selection: finds every informative feature, not top-K
- Grading students against a pigeon taking the same exam



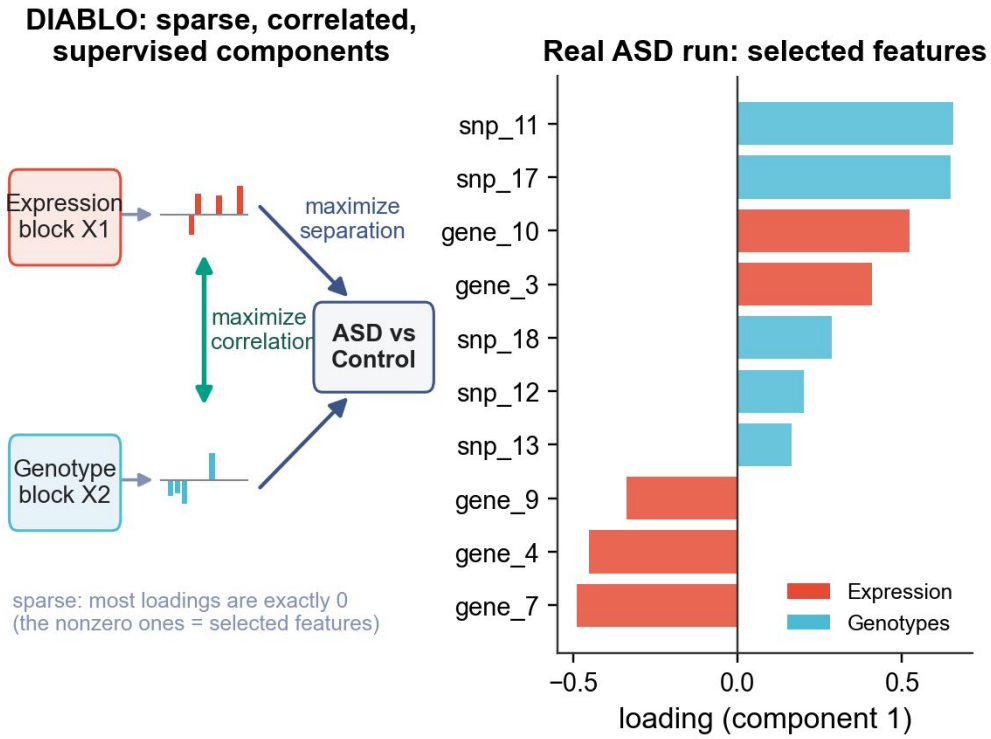
Method 13 - Stability Selection: Vote Across Resamples

- A panel selected once is a lottery ticket; selected 100x is evidence
- Recipe: subsample half the cohort → run L1 selection → repeat ~100x
- Keep features with selection frequency above ~60-80%
- Controls false discoveries under mild assumptions (Meinshausen 2010)
- Tames the lasso's correlated-twin instability
- The selection-side mirror of consensus clustering (Lesson 8)



Method 14 - DIABLO: Multi-Omics Selection (Embedded)

- The multi-omics specialist (mixOmics R package)
- Sparse components per block, built to CORRELATE across omics
    - AND to separate the classes (supervised)
- Selection = sparse loadings: few genes + few SNPs per component
- Output: matched multi-omics signatures, not two separate lists
- ASD run: 5 genes + 5 SNPs per component, block correlation 0.55



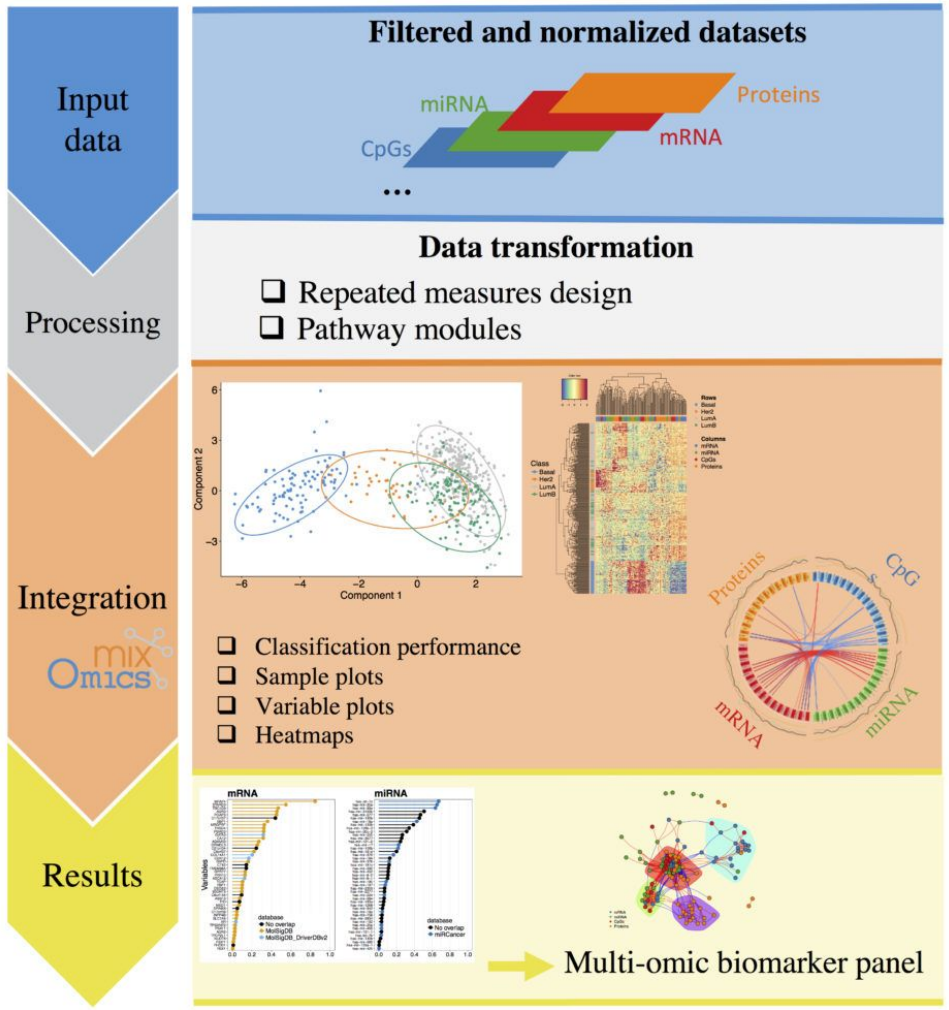
Method 14 - DIABLO (Data Integration Analysis for Biomarker Discovery)

Supervised integration of multiple datasets in relation to a categorical outcome variable

Employs multiblock sPLS-DA (sparse Partial Least Squares – Discriminant Analysis) to identify correlations between datasets

DIABLO constructs latent components by maximising the covariances between datasets, while balancing model discrimination and integration.

Enforcing sparsity means constraining the model so that only a predefined, small number of features per omics layer have non-zero weights, enabling feature selection, interpretability, and robust multi-omics integration.



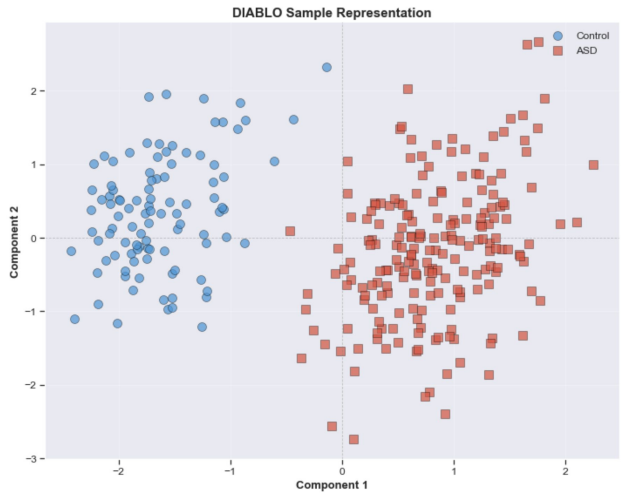
Method 14 - DIABLO - inputs and outputs

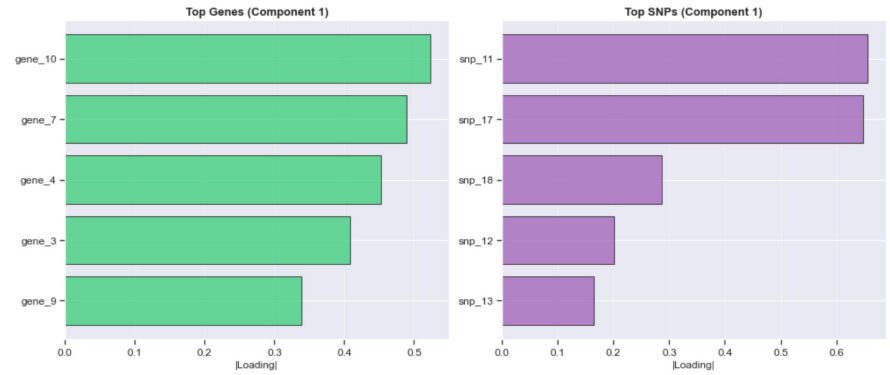
sample	gene_1	gene_2	gene_3	gene_4	gene_5
sample_1	14	179	217	620	27
sample_2	19	271	202	2277	53
sample_3	9	299	391	1102	33
sample_4	4	109	86	956	10
sample_5	4	141	90	1124	9

sample	snp_1	snp_2	snp_3	snp_4	snp_5
sample_1	0	0	0	0	1
sample_2	0	0	0	0	0
sample_3	0	0	0	0	0
sample_4	0	0	0	0	0
sample_5	0	0	0	0	0

sample	ASD
sample_1	1
sample_2	0
sample_3	1
sample_4	0
sample_5	0

sample	Comp1	Comp2
sample_1	0.508	-0.390
sample_2	-1.519	-0.819
sample_3	0.350	-1.230
sample_4	-0.861	1.600
sample_5	-2.050	1.043





Component	Block	Feature	Loading
1	Expression	gene_10	0.524
1	Expression	gene_7	-0.489
1	Expression	gene_4	-0.453
1	Expression	gene_3	0.408
1	Expression	gene_9	-0.339
1	Genotypes	snp_11	0.656
1	Genotypes	snp_17	0.648

DIABLO: Integrative Multi-Omics Feature Selection and Classification

1. Inputs

Transcriptomics (genes)



$n_1 \times p_1$

Proteomics (proteins)



$n_2 \times p_2$

Metabolomics (metabolites)



$n_3 \times p_3$

Outcome Y
(e.g. disease vs control)



2. Design Matrix

Specify how strongly components correlate across omics blocks

	Comp 1	Comp 2
Transcriptomics	1.0	0.5
Proteomics	1.0	0.5
Metabolomics	1.0	0.5

Controls the degree of multi-omics integration

3. Initialization

Use PLS-type directions to maximize covariance with Y for each block

W_{trans} W_{prot} W_{metab}

4. Iterative Optimization (Core of DIABLO)

1 Update weights for each block

Maximize a combination of:
- correlation with outcome Y
- correlation with other blocks

+ sparsity (soft-thresholding)

2 Normalize weights

Ensure comparable scales across blocks

3 Deflate data

Remove explained variation so next component captures complementarty info

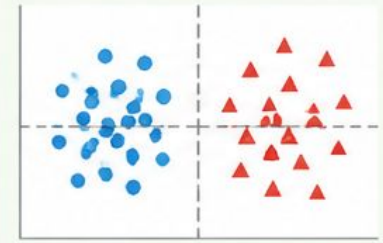
4 Repeat

Iterate across blocks until convergence (for each component)

5. Outputs

A. Predictive Model

Classify / predict new samples using integrated components



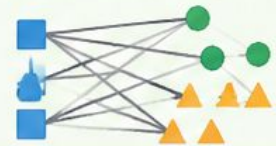
B. Selected Features

Top features per omics & component

	Comp 1	Comp 2
Genes	10	10
Proteins	8	6
Metabolites	5	4

C. Integrative Biomarker Signature

Coherent multi-omics set of informative features

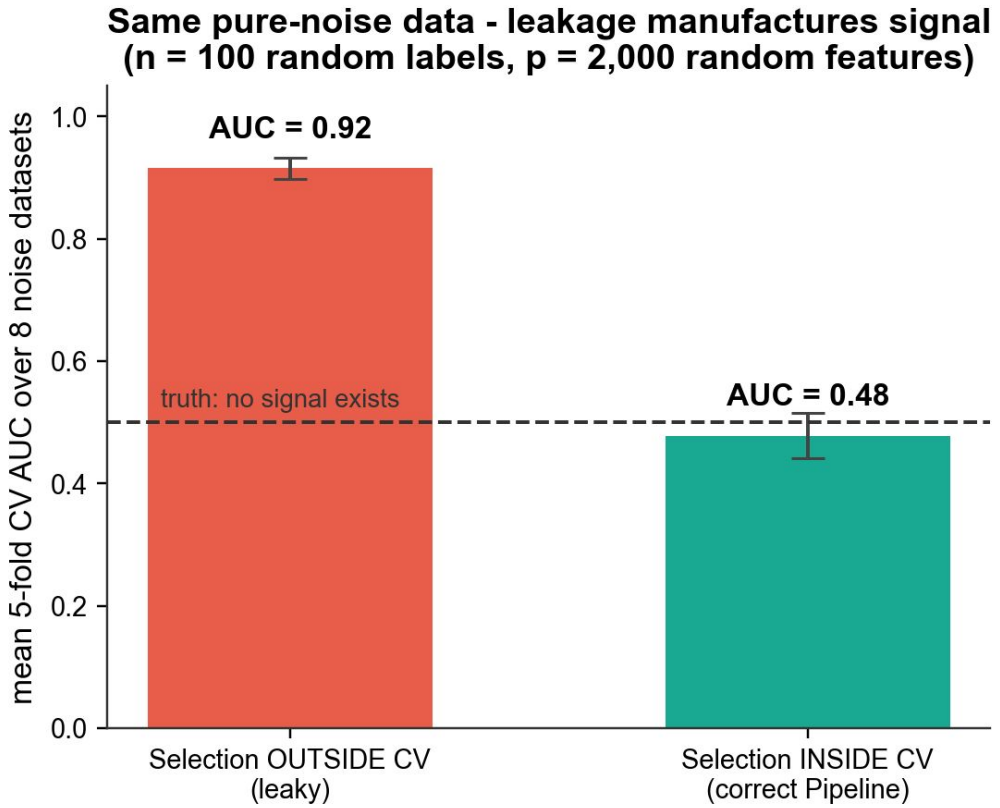


Key Points

1. Integrates multiple omics modalities via linked components.
2. Selects sparse sets of features per block and per component.
3. Iteratively optimizes and deflates to capture complementary multi-omics signals.
4. Delivers an interpretable multi-omics signature for prediction and discovery.

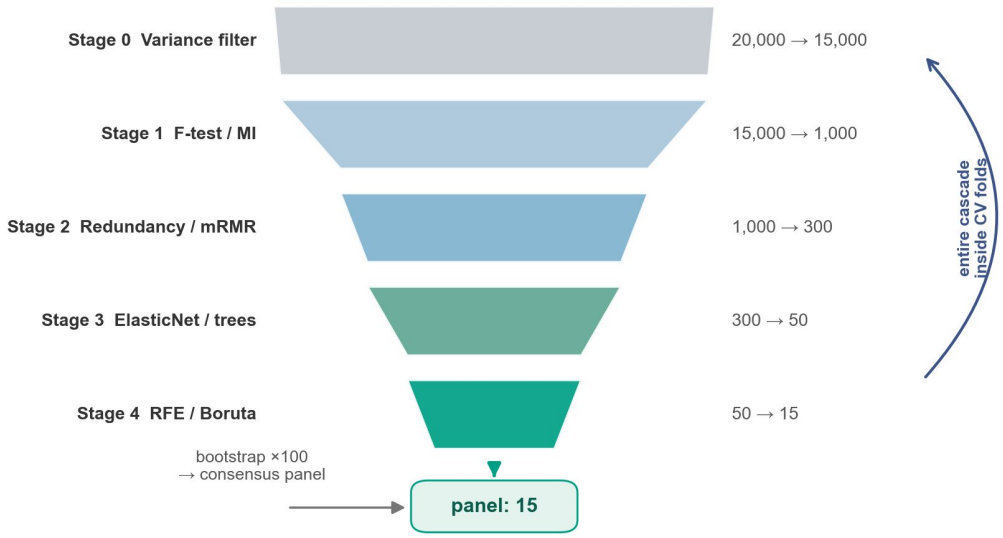
The Leakage Trap - Select Inside the CV Loop

- WRONG: select features on ALL data → then cross-validate
- The test folds already voted on which features survive!
- Consequence: AUC inflated, sometimes from 0.5 to 0.9 on pure noise
- RIGHT: selection re-fit on the training part of every fold
- sklearn Pipeline(selector, model) inside cross_val_score does this
- The single most common fatal flaw in omics ML papers



A Practical Multi-Stage Pipeline

- Stage 0 Variance filter: 20,000 → ~15,000 (free)
- Stage 1 Univariate F-test / FDR / MI: → ~1,000 (seconds)
- Stage 2 Redundancy pruning / mRMR: → ~300
- Stage 3 ElasticNet or tree SelectFromModel: → ~50
- Stage 4 RFE / SFS / Boruta refinement: → 10-20 panel
- Consensus across methods & bootstrap runs = robust biomarkers



Lesson 6 Summary

- Filters: variance, F-test, FDR control, MI, redundancy, mRMR
- Wrappers: RFE, sequential forward/backward, Boruta
- Embedded: Lasso/ElasticNet, group lasso, trees, DIABLO
- Correlated omics features demand redundancy handling
- Selection ALWAYS inside the CV loop - leakage fabricates AUC
- Stability selection and consensus separate biomarkers from lottery
- Next lesson: quantifying each feature's contribution - importances

